МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
КАФЕДРА ИИТ

Лабораторная работа №3

По дисциплине «Современный платформы программирования»

Выполнил:
Студент 3 курса
группы ПО-12:
Ефимович С.Л.
Проверил:
Кулик А.Д.

Брест 2026

Цель: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Ход работы

Задание 1 (Вариант 5) Первая группа заданий (порождающий паттерн): Завод по производству смартфонов. Обеспечить создание нескольких различных моделей мобильных телефонов с заранее выбранными характеристиками.

Код программы:

```
from abc import ABC, abstractmethod


class Smartphone:

   def __init__(self, model, processor, display, camera, body):
       self.model = model
       self.processor = processor
       self.display = display
       self.camera = camera
       self.body = body

   def show_info(self):
       print(f"\n--- {self.model} ---")
       print(f"Процессор: {self.processor}")
       print(f"Дисплей: {self.display}")
       print(f"Камера: {self.camera}")
       print(f"Корпус: {self.body}")


class PhoneFactory(ABC):

   @abstractmethod
   def create_processor(self):
       pass

   @abstractmethod
   def create_display(self):
       pass

   @abstractmethod
   def create_camera(self):
       pass

   @abstractmethod
   def create_body(self):
       pass

   @abstractmethod
   def get_model_name(self):
```

```
        pass


class BudgetFactory(PhoneFactory):

    def create_processor(self):
        return "MediaTek Helio G25, 2.0 ГГц"

    def create_display(self):
        return "6.1 дюймов, IPS, 1560x720"

    def create_camera(self):
        return "13 МП, вспышка, автофокус"

    def create_body(self):
        return "Пластик, черный, 185 г"

    def get_model_name(self):
        return "Galaxy A04"


class MidRangeFactory(PhoneFactory):

    def create_processor(self):
        return "Snapdragon 778G, 2.4 ГГц"

    def create_display(self):
        return "6.5 дюймов, AMOLED, 2400x1080"

    def create_camera(self):
        return "64 МП, стабилизация, 4K видео"

    def create_body(self):
        return "Стекло, синий, 175 г"

    def get_model_name(self):
        return "Galaxy A54"


class FlagshipFactory(PhoneFactory):

    def create_processor(self):
        return "Snapdragon 8 Gen 2, 3.2 ГГц"

    def create_display(self):
        return "6.8 дюймов, Dynamic AMOLED, 3200x1440, 120Гц"

    def create_camera(self):
        return "200 МП, лазерный фокус, 8K видео"

    def create_body(self):
        return "Титан, серебристый, 195 г"

    def get_model_name(self):
        return "Galaxy S23 Ultra"


class PhoneFactoryPlant:
```

```
    @staticmethod
    def create_phone(factory):
        return Smartphone(
            model=factory.get_model_name(),
            processor=factory.create_processor(),
            display=factory.create_display(),
            camera=factory.create_camera(),
            body=factory.create_body(),
        )


if __name__ == "__main__":
    print("ЗАВОД ПО ПРОИЗВОДСТВУ СМАРТФОНОВ")
    print("=" * 40)

    budget_phone = PhoneFactoryPlant.create_phone(BudgetFactory())
    mid_phone = PhoneFactoryPlant.create_phone(MidRangeFactory())
    flagship_phone = PhoneFactoryPlant.create_phone(FlagshipFactory())

    budget_phone.show_info()
    mid_phone.show_info()
    flagship_phone.show_info()

    print("\n" + "=" * 40)
    print("Производство завершено!")
```

Результат работы программы:

```
ЗАВОД ПО ПРОИЗВОДСТВУ СМАРТФОНОВ
========================================

--- Galaxy A04 ---
Процессор: MediaTek Helio G25, 2.0 ГГц
Дисплей: 6.1 дюймов, IPS, 1560x720
Камера: 13 МП, вспышка, автофокус
Корпус: Пластик, черный, 185 г

--- Galaxy A54 ---
Процессор: Snapdragon 778G, 2.4 ГГц
Дисплей: 6.5 дюймов, AMOLED, 2400x1080
Камера: 64 МП, стабилизация, 4K видео
Корпус: Стекло, синий, 175 г

--- Galaxy S23 Ultra ---
Процессор: Snapdragon 8 Gen 2, 3.2 ГГц
Дисплей: 6.8 дюймов, Dynamic AMOLED, 3200x1440, 120Гц
Камера: 200 МП, лазерный фокус, 8K видео
Корпус: Титан, серебристый, 195 г

========================================
Производство завершено!
```

Рисунок 1. Результат выполнения программы

Задание 2 (Вариант 5) Вторая группа заданий (структурный паттерн): Проект «Электронный градусник». В проекте должен быть реализован класс, который дает возможность пользоваться аналоговым градусником так же, как и электронным. В классе «Аналоговый градусник» хранится высота ртутного столба и границы измерений (верхняя и нижняя).

Код программы:

```
class AnalogThermometer:
   def __init__(self, min_temp, max_temp):
       self.min_temp = min_temp
       self.max_temp = max_temp
       self.height = 0

   def set_height(self, height):
       if 0 <= height <= 100:
           self.height = height

   def get_height(self):
       return self.height

   def get_range(self):
       return self.min_temp, self.max_temp


class ElectronicThermometer:
   def get_temperature(self):
       pass


class ThermometerAdapter(ElectronicThermometer):
   def __init__(self, analog_thermometer):
       self.analog = analog_thermometer

   def get_temperature(self):
       min_temp, max_temp = self.analog.get_range()
       height = self.analog.get_height()
       temperature = min_temp + (height / 100) * (max_temp - min_temp)
       return round(temperature, 1)


class DigitalThermometer(ElectronicThermometer):
   def __init__(self):
       self.temperature = 22.5

   def get_temperature(self):
       return self.temperature

   def set_temperature(self, temp):
       self.temperature = temp


if __name__ == "__main__":
   analog = AnalogThermometer(-20, 40)
   analog.set_height(75)
```

```
    adapter = ThermometerAdapter(analog)
    print(f"Аналоговый как электронный: {adapter.get_temperature()}°C")

    digital = DigitalThermometer()
    print(f"Цифровой: {digital.get_temperature()}°C")
```

Результат работы программы:

```
Аналоговый как электронный: 25.0°C
Цифровой: 22.5°C
```

Рисунок 2. Результат выполнения программы

Задание 3 (Вариант 5) Третья группа заданий (поведенческий паттерн): Проект «Банкомат». Предусмотреть выполнение основных операций (ввод пин-кода, снятие суммы, завершение работы) и наличие различных режимов работы (ожидание, аутентификация, выполнение операции, блокировка – если нет денег). Атрибуты: общая сумма денег в банкомате, ID.

Код программы:

```
class ATM:
    def __init__(self, atm_id, initial_money):
        self.id = atm_id
        self.total_money = initial_money
        self.card_inserted = False
        self.authenticated = False
        self.state = WaitingState(self)

    def set_state(self, state):
        self.state = state

    def insert_card(self):
        self.state.insert_card()

    def enter_pin(self, pin):
        self.state.enter_pin(pin)

    def withdraw(self, amount):
        self.state.withdraw(amount)

    def cancel(self):
        self.state.cancel()

    def get_state_name(self):
        return self.state.__class__.__name__
```

```
class ATMState:
   def __init__(self, atm):
       self.atm = atm

   def insert_card(self):
       print("Операция недоступна в текущем режиме")

   def enter_pin(self, pin):
       print("Операция недоступна в текущем режиме")

   def withdraw(self, amount):
       print("Операция недоступна в текущем режиме")

   def cancel(self):
       print("Операция недоступна в текущем режиме")


class WaitingState(ATMState):
   def insert_card(self):
       print("Карта вставлена. Введите пин-код")
       self.atm.card_inserted = True
       self.atm.set_state(PinAuthState(self.atm))


class PinAuthState(ATMState):
   def __init__(self, atm):
       super().__init__(atm)
       self.attempts = 0

   def enter_pin(self, pin):
       if pin == "1234":
           print("Пин-код верный")
           self.atm.authenticated = True
           self.atm.set_state(OperationState(self.atm))
       else:
           self.attempts += 1
           print(f"Неверный пин-код. Попытка {self.attempts}/3")
           if self.attempts >= 3:
               print("Слишком много попыток. Карта заблокирована")
               self.atm.set_state(BlockedState(self.atm))

   def cancel(self):
       print("Операция отменена. Заберите карту")
       self.atm.card_inserted = False
       self.atm.set_state(WaitingState(self.atm))


class OperationState(ATMState):
   def withdraw(self, amount):
       if amount <= 0:
           print("Сумма должна быть положительной")
           return

       if amount > self.atm.total_money:
           print(f"Недостаточно средств в банкомате. Доступно:
{self.atm.total_money}")
           self.atm.set_state(BlockedState(self.atm))
           return
```

```
        if amount > 10000:
            print("Сумма превышает лимит на одну операцию")
            return

        self.atm.total_money -= amount
        print(f"Выдано {amount} руб. Остаток в банкомате: {self.atm.total_money}")

        if self.atm.total_money == 0:
            print("В банкомате закончились деньги")
            self.atm.set_state(BlockedState(self.atm))

    def cancel(self):
        print("Сеанс завершен. Заберите карту")
        self.atm.card_inserted = False
        self.atm.authenticated = False
        self.atm.set_state(WaitingState(self.atm))


class BlockedState(ATMState):
    def insert_card(self):
        print("Банкомат временно не работает. Попробуйте позже")

    def enter_pin(self, pin):
        print("Банкомат временно не работает")

    def withdraw(self, amount):
        print("Банкомат временно не работает")

    def cancel(self):
        print("Возврат в начальный режим")
        self.atm.card_inserted = False
        self.atm.authenticated = False
        self.atm.set_state(WaitingState(self.atm))


if __name__ == "__main__":
    atm = ATM("ATM-001", 50000)

    print(f"Банкомат {atm.id} запущен. Денег: {atm.total_money}")
    print("-" * 40)

    atm.insert_card()
    atm.enter_pin("1111")
    atm.enter_pin("2222")
    atm.enter_pin("1234")

    atm.withdraw(5000)
    atm.withdraw(20000)
    atm.withdraw(30000)

    atm.cancel()
    print(f"Состояние: {atm.get_state_name()}")

    print("\n" + "-" * 40)
    print("Тест блокировки при отсутствии денег:")

    atm.insert_card()
    atm.enter_pin("1234")
```

```
atm.withdraw(25000)
atm.withdraw(1000)
atm.cancel()
```

Результат работы программы:

```
Банкомат ATM-001 запущен. Денег: 50000
----------------------------------------
Карта вставлена. Введите пин-код
Неверный пин-код. Попытка 1/3
Неверный пин-код. Попытка 2/3
Пин-код верный
Выдано 5000 руб. Остаток в банкомате: 45000
Сумма превышает лимит на одну операцию
Сумма превышает лимит на одну операцию
Сеанс завершен. Заберите карту
Состояние: WaitingState

----------------------------------------
Тест блокировки при отсутствии денег:
Карта вставлена. Введите пин-код
Пин-код верный
Сумма превышает лимит на одну операцию
Выдано 1000 руб. Остаток в банкомате: 44000
Сеанс завершен. Заберите карту
```

Рисунок 3. Результат выполнения программы

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python